

## Módulo 6 – Capítulo 05 – Sección 01

---

### Búsqueda semántica vs búsqueda léxica (BM25): fortalezas y limitaciones de cada una

Imagina dos sistemas de recuperación distintos enfrentando la misma consulta: "medicamentos para el dolor de articulaciones". El sistema de búsqueda léxica basado en BM25 buscará documentos que contengan literalmente las palabras "medicamentos", "dolor" y "articulaciones" y los ordenará por frecuencia y relevancia estadística de esos términos. El sistema de búsqueda semántica basada en embeddings buscará documentos cuyo vector sea cercano en el espacio latente al vector de la query, encontrando también documentos sobre "antiinflamatorios para la artritis" o "fármacos AINE para reumatología" aunque no compartan ninguna palabra con la consulta. Ahora imagina una consulta diferente: "ECONNREFUSED error en Node.js 18.4.2". Aquí el sistema semántico puede producir resultados incorrectos porque el modelo de embedding puede no representar bien ese identificador técnico específico, mientras que BM25 lo encuentra exactamente porque es una búsqueda de tokens literales. Esta asimetría de fortalezas es fundamental: no existe un sistema de recuperación dominante para todos los tipos de queries.

**BM25 (Best Match 25)** es un algoritmo de ranking probabilístico que extiende el modelo TF-IDF clásico con dos mejoras: saturación de la frecuencia de términos (la contribución de un término al score crece logarítmicamente con su frecuencia, no linealmente) y normalización por longitud del documento (documentos más largos no tienen ventaja artificial sobre documentos más cortos con la misma densidad de términos relevantes). Elasticsearch, OpenSearch y BM25S son las implementaciones más utilizadas en producción. BM25 tiene varias propiedades que lo hacen indispensable en sistemas de recuperación empresariales: es **determinista** (la misma query produce siempre el mismo ranking), es **interpretable** (el score puede descomponerse por término para entender por qué un documento fue recuperado), es **independiente de cualquier modelo** (no requiere GPU ni llamadas a APIs externas) y es **especialmente preciso** para queries con términos específicos que deben aparecer literalmente en el documento —nombres propios, identificadores de producto, números de contrato, códigos de error, términos de arte legal.

La **búsqueda semántica** basada en embeddings vectoriales tiene el perfil de fortalezas opuesto. Su capacidad de recuperar documentos semánticamente relacionados con la query aunque no compartan ninguna palabra la hace ideal para queries en lenguaje natural donde el vocabulario del usuario puede diferir del vocabulario del corpus. Un médico que pregunta "¿cómo se trata la inflamación crónica de las articulaciones?" puede encontrar documentos sobre "terapia con DMARDs para artritis reumatoide" aunque no use ninguno de esos términos. Esta capacidad de recuperación cross-vocabulario es especialmente valiosa en sistemas donde los usuarios son no expertos consultando un corpus de documentación técnica o especializada. Sin embargo, la búsqueda semántica tiene limitaciones sistemáticas que BM25 no comparte: puede producir resultados incorrectos para consultas con identificadores técnicos específicos (modelos de máquinas, números de versión, identificadores únicos) porque esos tokens raramente aparecen con alta frecuencia en el corpus de entrenamiento del modelo de embedding, haciendo que su representación vectorial sea imprecisa.

La complementariedad de ambos enfoques es la razón por la que los sistemas de recuperación más robustos de producción no eligen uno u otro: los combinan. Un sistema que solo implementa búsqueda semántica tiene un techo de calidad sistemáticamente inferior al que implementa ambas, porque las queries que BM25 maneja mejor (términos específicos, identificadores, nombres propios) son exactamente las que la búsqueda semántica maneja peor, y viceversa. El mecanismo de combinación —la recuperación híbrida— es el tema de la siguiente sección.

## Fortalezas y limitaciones técnicas comparadas

- **BM25 fortalezas:** matching exacto de términos; funciona para nombres propios, identificadores, códigos, números de versión; sin overhead de modelo de embedding en inferencia; determinista y reproducible; implementado eficientemente con índices invertidos (Elasticsearch, OpenSearch, BM25S).
- **BM25 limitaciones:** sin comprensión semántica; no recupera sinónimos ni paráfrasis; sensible al vocabulario exacto ("auto" vs. "coche" vs. "vehículo" son términos distintos); falla cuando la query usa terminología diferente al corpus aunque el contenido sea el mismo.
- **Búsqueda semántica fortalezas:** recuperación por similitud conceptual independiente del vocabulario; maneja sinónimos, paráfrasis, reformulaciones; captura la intención semántica de queries complejas; soporta queries multilingüe con modelos como BGE-M3 o voyage-multilingual.

- **Búsqueda semántica limitaciones:** falla con tokens OOV (out-of-vocabulary) del corpus de entrenamiento del embedding; puede confundir identifiers similares ("GPT-4" vs. "GPT-4o" vs. "GPT-4 turbo"); más costosa computacionalmente en indexación por las llamadas al modelo de embedding.
- **Queries que favorecen BM25:** "contrato número 2024-LAT-0892", "error ECONNREFUSED en Node.js", "ivermectina 6mg dosis adultos", "cláusula de fuerza mayor artículo 15.3"; cualquier query con tokens específicos que deban aparecer literalmente en el resultado.
- **Queries que favorecen búsqueda semántica:** "¿cuáles son los efectos secundarios del tratamiento?", "explícame cómo funciona el proceso de aprobación", "ventajas de esta tecnología sobre las alternativas"; queries conceptuales donde el vocabulario de la respuesta puede diferir del de la query.

**Nota del Arquitecto:** En sistemas para usuarios técnicos (developers, abogados, médicos), el porcentaje de queries léxicamente específicas es mucho mayor que en sistemas para usuarios generales. Un asistente de código recibe mayoritariamente queries con nombres de funciones, errores exactos y versiones de librerías; necesita BM25 con más peso que el semántico. Un asistente de recursos humanos recibe queries conceptuales sobre políticas y beneficios; necesita el semántico con más peso. El  $\alpha$  óptimo de la fusión híbrida (próxima sección) es específico del dominio y de la distribución de queries de cada sistema.

El reconocimiento de que BM25 y la búsqueda semántica tienen perfiles de fortaleza complementarios lleva directamente a la pregunta de cómo combinar sus resultados de forma efectiva. La siguiente sección analiza los algoritmos de fusión —RRF y weighted score fusion— y cómo implementar la recuperación híbrida en producción.

## Módulo 6 – Capítulo 05 – Sección 02

### Recuperación híbrida: combinar dense y sparse retrieval con RRF y puntuaciones ponderadas

La recuperación híbrida no es simplemente "hacer dos búsquedas y juntar los resultados": requiere un algoritmo de fusión que combine dos listas de resultados con escalas de score completamente distintas (la similitud coseno del retriever vectorial produce valores en  $[-1, 1]$ ; el score BM25 produce valores en un rango no normalizado que depende de la longitud del corpus y de los documentos) en una única lista ordenada que capture lo mejor de ambos enfoques. Existen dos algoritmos principales para esta fusión, con perfiles de complejidad y calibración distintos: Reciprocal Rank Fusion (RRF) y weighted score fusion.

**Reciprocal Rank Fusion (RRF)**, propuesto por Cormack et al. (2009), es el algoritmo de fusión más adoptado en producción por su robustez y simplicidad de implementación. La fórmula es elegantemente simple: para cada documento, calcular la suma de  $\frac{1}{k + \text{rank}_i}$  para cada lista donde aparece, donde  $\text{rank}_i$  es la posición del documento en la lista  $i$  y  $k=60$  es un hiperparámetro de suavizado que reduce la influencia desproporcionada de los documentos en la primera posición. El documento que aparece en posición 1 en ambas listas recibe un score de  $\frac{2}{(60+1)} \approx 0.033$ ; el que aparece en posición 5 en ambas listas recibe  $\frac{2}{(60+5)} \approx 0.031$ ; el que aparece solo en posición 1 en una lista recibe  $\frac{1}{61} \approx 0.016$ . La ventaja fundamental de RRF es que opera **sobre rangos, no sobre scores**: no necesita normalizar los scores del retriever vectorial y del BM25 porque solo considera la posición de cada documento en cada lista. Esto elimina el problema de incompatibilidad de escalas entre los dos retrievers y hace que RRF sea robusto ante outliers (un documento con un score BM25 muy alto no domina la fusión porque su contribución está limitada por la función  $1/(k+\text{rank})$ ).

La **weighted score fusion** es la alternativa cuando se quiere controlar explícitamente la contribución relativa de cada retriever. El mecanismo es lineal: normalizar los scores de cada retriever a  $[0,1]$  con min-max scaling ( $\text{score\_norm} = (\text{score} - \text{min}) / (\text{max} - \text{min})$ ) y combinarlos con un coeficiente alpha:  $\text{score\_final} = \text{alpha} \times \text{score\_semántico} + (1-\text{alpha}) \times \text{score\_BM25}$ . Con  $\text{alpha}=0.5$ , ambos retrievers contribuyen por igual; con  $\text{alpha}=0.7$ , el semántico tiene más peso; con  $\text{alpha}=0.3$ , BM25 domina. El valor de alpha es un

hiperparámetro que debe calibrarse mediante evaluación sobre el dataset interno: ejecutar la búsqueda híbrida con  $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$  y medir  $\text{Recall@5}$  para cada valor. La desventaja respecto a RRF es que la normalización min-max de los scores es sensible a outliers y puede ser inestable entre consultas con distribuciones de scores muy distintas.

**SPLADE (SParse Lexical AnD Expansion)** es una alternativa al BM25 clásico como retriever léxico en pipelines híbridos. SPLADE produce vectores dispersos (sparse vectors) donde cada dimensión corresponde a un término del vocabulario del modelo de lenguaje y el valor de la dimensión representa la importancia del término en el documento; la mayoría de las dimensiones son cero, de ahí el "disperso". La ventaja sobre BM25 es que SPLADE incorpora expansión de vocabulario: el modelo puede asignar importancia a términos que no aparecen literalmente en el documento pero que son semánticamente relacionados, capturando algo de la inteligencia semántica del modelo de lenguaje mientras mantiene la eficiencia de los índices invertidos. `naver-splade-v3` es el modelo SPLADE más usado en producción. Qdrant soporta nativamente vectores dispersos SPLADE junto a vectores densos en el mismo payload, permitiendo búsqueda híbrida sin un sistema BM25 externo.

La implementación práctica de la recuperación híbrida requiere mantener dos índices: el índice vectorial denso (en Qdrant, Pinecone o Weaviate) y el índice invertido para BM25 o SPLADE (en Elasticsearch, OpenSearch, o en Qdrant con vectores esparsos). Weaviate implementa la búsqueda híbrida nativamente en un único sistema con su parámetro  $\alpha$ . La clase `EnsembleRetriever` de LangChain permite combinar cualquier par de retrievers (incluyendo `BM25Retriever` y `VectorStoreRetriever`) con RRF en pocas líneas de código.

## Aspectos técnicos de la recuperación híbrida

- **Reciprocal Rank Fusion (RRF):** `RRF(d) = sum_i (1/(k+r_i))`;  $k=60$  recomendado por el paper original; opera sobre rangos, no scores; no requiere normalización; robusto ante outliers; el algoritmo de fusión correcto por defecto para la mayoría de los sistemas.
- **Weighted score fusion:** normalizar a  $[0,1]$  con min-max scaling y combinar con  $\alpha$ ; requiere calibración del  $\alpha$  sobre el dataset de evaluación;  $\alpha$  óptimo varía por dominio ( $\alpha > 0.6$  para corpus conceptual,  $\alpha < 0.4$  para corpus con muchos identificadores específicos).
- **SPLADE:** embeddings dispersos con expansión de vocabulario; mejor que BM25 para vocabulario técnico variado; compatible con índices invertidos eficientes; disponible como `naver-splade-v3` en Hugging Face; soportado nativamente en Qdrant como sparse vector.

- **EnsembleRetriever (LangChain):** combina múltiples retrievers con RRF; acepta `BM25Retriever` (usa rank-bm25 internamente) y `VectorStoreRetriever`; configuración de pesos por retriever; la opción más simple para prototipado de búsqueda híbrida.
- **Búsqueda híbrida nativa en Weaviate:** parámetro alpha en la query controla balance vectorial/BM25; el motor ejecuta ambas búsquedas internamente y aplica fusión; latencia ligeramente mayor que búsqueda individual por el overhead del merge; ideal para equipos que prefieren un único sistema.
- **Costos adicionales de la hibridación:** mantener el índice vectorial (en RAM o GPU) y el índice invertido BM25 (en disco con I/O eficiente); Elasticsearch + Qdrant o Weaviate nativo son las opciones que mejor integran ambos índices en un sistema operacional único.

La recuperación híbrida debe ser la configuración estándar de producción, no una optimización avanzada: el costo incremental de mantener un índice BM25 junto al vectorial es marginal comparado con la ganancia sistemática de Recall@K en queries léxicamente específicas. Con el retriever configurado correctamente, la siguiente sección analiza cómo el reranking puede elevar la Precision@K de los chunks que finalmente recibe el LLM generador.

---

## Módulo 6 – Capítulo 05 – Sección 03

---

### Reranking: modelos cross-encoder, Cohere Rerank, BGE Reranker

El retriever —ya sea semántico, BM25 o híbrido— opera con bi-encoders: modelos que codifican la query y el documento por separado, generan un vector para cada uno, y miden la similitud entre esos vectores. Esta arquitectura es eficiente (los vectores de documentos pueden pre-computarse en indexación y solo el vector de la query necesita computarse en tiempo de consulta) pero tiene una limitación fundamental: el modelo nunca ve la query y el documento simultáneamente, por lo que no puede capturar interacciones semánticas finas entre los tokens de ambos textos. El reranking resuelve esta limitación con una segunda etapa de evaluación más precisa pero más costosa: los modelos cross-encoder reciben la concatenación de query y documento como entrada y producen un score de relevancia que captura las interacciones entre todos los tokens de ambos textos.

La **arquitectura cross-encoder** es estructuralmente la misma que la de un modelo BERT de clasificación: recibe la secuencia `[CLS] query [SEP] documento [SEP]` y produce un score escalar de relevancia. La diferencia con el bi-encoder es que la atención cruzada entre los tokens de la query y los del documento permite al modelo capturar relaciones semánticas que son invisibles cuando ambos textos se procesan independientemente. Por ejemplo, si la query pregunta "¿cuál es la dosis máxima?" y el documento menciona "no superar los 3000mg diarios", el cross-encoder puede capturar que "dosis máxima" y "no superar los 3000mg" son semánticamente equivalentes en ese contexto, mientras que un bi-encoder puede asignar scores de similitud bajos porque los tokens superficialmente no se parecen. Esta mayor precisión tiene un costo: el cross-encoder requiere una pasada completa del transformer por cada par (query, documento), lo que hace imposible pre-computar los scores en indexación; el reranking debe ejecutarse en tiempo de consulta sobre el conjunto de chunks recuperados.

El **pipeline two-stage** es el patrón estándar en producción: la primera etapa usa el retriever rápido (vectorial o híbrido) para recuperar top-100 candidatos; la segunda etapa usa el cross-encoder para reranear esos 100 candidatos y seleccionar los top-5 o top-10 para el contexto del LLM. Este diseño mantiene el Recall@100 de la primera etapa (alta probabilidad de que el

documento correcto esté entre los 100 candidatos) mientras eleva la Precision@5 de la segunda etapa (los 5 chunks finales tienen alta probabilidad de ser los más relevantes). La mejora de Precision@5 con reranking es consistentemente de 15–25 puntos porcentuales sobre la primera etapa, lo que justifica el overhead de latencia adicional de 200–500ms que el reranker introduce.

**Cohere Rerank v3** es el modelo de reranking como servicio más utilizado en producción empresarial. La API acepta la query y hasta 1000 documentos, devuelve scores de relevancia normalizados de 0 a 1 para cada documento, y soporta más de 100 idiomas. El precio de \$2 por millón de documentos rankeados hace que el reranking de 100 chunks por consulta cueste \$0.0002 por consulta —un costo operacional mínimo comparado con el costo de los tokens del LLM generador. Cohere también ofrece un modelo lite a \$0.10/millón para casos con presupuesto más ajustado, con una reducción de calidad moderada.

**BGE Reranker v2-m3** (BAAI) es la alternativa open source de referencia: un modelo cross-encoder de 568 millones de parámetros con soporte multilingüe, licencia Apache 2.0, y rendimiento comparable a Cohere Rerank en benchmarks de BEIR. Puede desplegarse en GPU con ~2GB VRAM (una GPU T4 es suficiente) y ejecutar reranking de 100 chunks en ~200ms, eliminando los costos de API y las dependencias externas. Para equipos con restricciones de soberanía de datos o presupuesto de API limitado, BGE Reranker es la primera opción.

**FlashRank** es una librería Python que implementa modelos cross-encoder pequeños (ms-marco-MiniLM-L6-v2, ms-marco-TinyBERT-L6) optimizados para CPU con latencias de 5–20ms para 100 documentos. Es la opción correcta cuando GPU no está disponible en el serving o cuando el presupuesto de latencia es muy ajustado: un reranker de 15ms en CPU añade latencia mínima al pipeline y puede ejecutarse en instancias estándar sin GPU especializada.

## Componentes técnicos del reranking

- **Arquitectura cross-encoder:** entrada `[CLS] query [SEP] documento [SEP]`; atención cruzada entre tokens de query y documento; un score de relevancia por par; O(K) inferencias del modelo por consulta; precisión significativamente superior al bi-encoder pero sin posibilidad de pre-computar.
- **Latencia del reranking:** BGE Reranker en GPU V100: ~200ms para 100 documentos; Cohere Rerank API: 200–500ms para hasta 100 documentos; FlashRank en CPU: 5–

20ms para 100 documentos con modelos pequeños; el overhead de latencia siempre es justificado si la mejora de Precision@5 supera los 10 puntos.

- **Pipeline two-stage:** first-stage → top-100 con retriever rápido (vectorial o híbrido); second-stage → top-5/10 con cross-encoder; mantiene Recall@100 de la primera etapa mientras eleva Precision@5 de la segunda; mejora típica de 15–25 puntos en Precision@5.
- **Cohere Rerank v3:** 100+ idiomas; hasta 1000 documentos por llamada; latencia <500ms para 100 docs; precio \$2/millón de unidades; modelo lite a \$0.10/millón; opción managed para equipos sin GPU dedicada.
- **BGE Reranker v2-m3:** 568M parámetros; Apache 2.0; contexto de 8192 tokens; ~2GB VRAM; latencia ~200ms en GPU T4 para 100 docs; disponible en Hugging Face; desplegable con sentence-transformers o TGI.
- **FlashRank:** librería Python para reranking en CPU; modelos MiniLM y TinyBERT; 5–20ms para 100 docs en CPU; calidad inferior a BGE o Cohere pero suficiente para casos de uso con presupuesto de latencia muy ajustado.

**Nota del Arquitecto:** El reranking es la optimización de mayor ROI en cualquier sistema RAG porque trabaja sobre los chunks ya recuperados por el retriever y los reordena sin coste adicional de embedding ni de consulta a la base vectorial. En un sistema con Recall@100 de 0.90 pero Precision@5 de 0.45 (de los 100 chunks recuperados, solo 45% de los 5 seleccionados para el LLM son relevantes), añadir BGE Reranker eleva típicamente la Precision@5 a 0.70–0.80. Ese delta de 25–35 puntos en Precision@5 se traduce directamente en mejor faithfulness y answer relevancy del sistema completo.

Con retrieval híbrido y reranking configurados, el pipeline de recuperación tiene calidad de producción. La siguiente sección añade la última capa de optimización del lado de la query: las técnicas de transformación que mejoran el vector de consulta antes de enviarlo al retriever.

## Módulo 6 – Capítulo 05 – Sección 04

---

### Transformación de consultas: HyDE, multi-query y descomposición de preguntas

Existe una asimetría semántica fundamental entre los textos del corpus y las queries de los usuarios que ningún modelo de embedding puede eliminar completamente: los documentos del corpus están escritos en el estilo del experto que los produjo (formal, técnico, denso), mientras que las queries de los usuarios están escritas en su propio lenguaje (informal, incompleto, impreciso). Un médico que escribe en la historia clínica "episodio de taquicardia ventricular sostenida con inestabilidad hemodinámica" y un usuario que pregunta "¿qué pasa cuando el corazón late muy rápido?" son semánticamente equivalentes pero léxicamente muy distintos. Las técnicas de transformación de consultas abordan esta asimetría actuando en el lado de la query, antes de la búsqueda vectorial, para producir un vector de consulta que esté más cerca en el espacio latente de los documentos relevantes del corpus.

**HyDE (Hypothetical Document Embeddings)**, propuesto por Gao et al. (2022), invierte el problema de forma contraintuitiva: en lugar de buscar el embedding de la query del usuario, pide al LLM que genere un documento hipotético que respondería esa query, y usa el embedding del documento hipotético como vector de búsqueda. La intuición es poderosa: el documento hipotético generado por el LLM usa el vocabulario, la estructura y el estilo del tipo de documento que respondería la query —que son los mismos del corpus real— produciendo un vector que está mucho más cerca en el espacio latente de los documentos relevantes que el vector de la query original. Si el usuario pregunta "¿cuáles son los efectos del ibuprofeno en pacientes con insuficiencia renal?", el LLM genera un párrafo hipotético como "El ibuprofeno, un AINE inhibidor de COX-1 y COX-2, puede exacerbar la función renal en pacientes con insuficiencia renal crónica mediante la reducción de la síntesis de prostaglandinas renales vasodilatadoras..."; el embedding de ese párrafo hipotético está mucho más cerca del artículo médico real sobre el tema que el embedding de la query original. HyDE mejora Recall@5 en 5–15 puntos para queries abstractas o ambiguas, pero puede ser contraproducente para queries factuales muy específicas donde el LLM puede alucinar datos incorrectos en el documento hipotético.

El **multi-query retrieval** (`MultiQueryRetriever` en LangChain) ataca el problema desde un ángulo diferente: en lugar de transformar la query en un único vector mejor, genera 3–5 reformulaciones distintas de la query original usando el LLM y ejecuta búsquedas separadas para cada una. Las reformulaciones pueden explorar diferentes ángulos de la query (la pregunta desde la perspectiva del fabricante, desde la del usuario, en terminología técnica, en terminología coloquial), sinónimos del vocabulario principal, o formulaciones más específicas o más generales. Los resultados de todas las búsquedas se combinan con deduplicación por ID de chunk, produciendo un conjunto de candidatos más diverso y con mayor cobertura del espacio semántico relevante. El costo es multiplicar el número de llamadas al retriever por el número de reformulaciones (3–5x más llamadas), más el costo de la llamada al LLM para generar las reformulaciones (~100ms adicionales).

El **step-back prompting** es una técnica complementaria que opera en un nivel de abstracción diferente: antes de buscar la respuesta a la pregunta específica, el sistema pide al LLM que identifique el concepto o principio general que subyace a la pregunta, y realiza una búsqueda adicional sobre ese concepto general. Si el usuario pregunta "¿A qué temperatura hierve el agua a 3.500 metros de altitud?", el step-back identifica el principio general "relación entre presión atmosférica y punto de ebullición del agua" y recupera contexto de fondo sobre ese principio antes de buscar la respuesta específica a la altitud. Esto mejora la calidad de la generación cuando la respuesta requiere entender el marco teórico general, no solo acceder a un dato puntual.

La **descomposición de consultas** (query decomposition) aborda el caso de queries compuestas que requieren información de múltiples fuentes: "¿Cuáles son las diferencias y similitudes entre el protocolo A y el protocolo B en términos de seguridad, rendimiento y costo?". Responder esta query con una única búsqueda vectorial es difícil porque la pregunta es multidimensional: requiere información sobre el protocolo A (seguridad, rendimiento, costo), sobre el protocolo B (las mismas dimensiones) y luego síntesis comparativa. La descomposición divide esta query en sub-queries atómicas ("¿cuáles son las características de seguridad del protocolo A?", "¿cuáles son las características de seguridad del protocolo B?", etc.), ejecuta RAG para cada sub-query independientemente, y sintetiza las respuestas parciales en una respuesta final integrada.

## Técnicas de transformación de consultas

- **HyDE (Hypothetical Document Embeddings):** LLM genera un documento hipotético de 100–200 palabras que respondería la query → embedir el documento

hipotético → usar su vector para la búsqueda; mejora Recall@5 en 5–15% para queries abstractas; overhead de ~500ms por una inferencia LLM; no funciona bien para queries factuales donde el LLM puede alucinar datos en el hipotético.

- **Multi-query retrieval (MultiQueryRetriever):** LLM genera 3–5 reformulaciones de la query → búsqueda separada para cada reformulación → deduplicar resultados por ID de chunk → unión de todos los resultados únicos; mejora Recall@K a costa de 3–5x más llamadas al retriever; latencia adicional de 100–200ms por la generación de reformulaciones.
- **Step-back prompting:** LLM identifica el concepto o principio general detrás de la query específica → búsqueda del concepto general + búsqueda de la query original → combinación de resultados; mejora la calidad de la generación cuando la respuesta requiere contexto de fondo conceptual.
- **Query decomposition:** LLM descompone queries compuestas en sub-queries atómicas → RAG independiente para cada sub-query → síntesis de respuestas parciales; adecuado para queries de comparación, contraste o síntesis multi-documento; latencia multiplicada por el número de sub-queries.
- **Corrective query expansion:** analizar los chunks recuperados en la primera búsqueda, identificar términos técnicos relevantes ausentes de la query original, reformular incorporando esos términos y ejecutar una segunda búsqueda; implementable con un LLM pequeño como clasificador de relevancia de términos.
- **Costo-beneficio de la transformación:** cada técnica añade latencia (HyDE: +500ms; multi-query: +300ms + 2–5x llamadas al retriever) y costo de tokens LLM; evaluar el delta de Recall@K sobre el dataset de evaluación para determinar si el overhead es justificado en el caso de uso específico.

**Nota del Arquitecto:** Las técnicas de transformación de consultas son especialmente valiosas cuando hay un vocabulario mismatch grande entre los usuarios del sistema y el corpus. En un sistema RAG para técnicos de mantenimiento industrial que consulta documentación de ingeniería, los técnicos usan lenguaje coloquial ("la máquina hace ruido") mientras la documentación usa terminología técnica ("vibración anómala en el conjunto del rodamiento principal"). Multi-query con reformulaciones que incluyen la terminología técnica equivalente resuelve este gap. En sistemas donde el vocabulario del usuario y del corpus son similares, el overhead no está justificado.

Las técnicas de transformación de consultas completan el arsenal de optimización del retriever: búsqueda híbrida para cobertura semántica y léxica, reranking para precisión en los chunks finales, y transformación de queries para bridgear el vocabulario gap entre usuarios y corpus. La siguiente sección cierra el análisis del retriever con las métricas que permiten medir y monitorear su calidad de forma sistemática.

---

## Módulo 6 – Capítulo 05 – Sección 05

---

### Métricas de recuperación: Recall@K, MRR, NDCG

Las optimizaciones del retriever descritas en las secciones anteriores —búsqueda híbrida, reranking, transformación de queries— solo pueden evaluarse objetivamente si existe un conjunto de métricas cuantitativas medidas sobre datos representativos. Sin métricas, cualquier cambio en el pipeline de recuperación tiene efectos desconocidos: puede mejorar el Recall para algunas queries y degradarlo para otras, sin que el equipo pueda detectar el delta neto. Las métricas de recuperación son el instrumento de medición que convierte la optimización del retriever de un proceso de intuición en un proceso de ingeniería.

**Recall@K** es la métrica principal para sistemas RAG. La definición es directa: para cada query del dataset de evaluación,  $\text{Recall@K} = (\text{número de documentos relevantes recuperados entre los top-K}) / (\text{número total de documentos relevantes para esa query})$ . En el caso más común en sistemas RAG empresariales —donde hay exactamente un chunk que contiene la respuesta correcta por query— Recall@K se simplifica a "¿está el chunk correcto entre los K recuperados?": un resultado binario de 0 o 1 por query, promediado sobre todo el dataset. Un Recall@5 de 0.75 significa que para el 75% de las queries del dataset, el chunk con la respuesta correcta está entre los cinco primeros recuperados; para el 25% restante, el retriever ha fallado y el LLM generador no tendrá la información necesaria para responder correctamente, independientemente de su capacidad de razonamiento. Este es el vínculo directo entre el Recall@K del retriever y la calidad de las respuestas del sistema: un retriever con Recall@5 de 0.60 garantiza que el 40% de las respuestas del sistema estarán incorrectas o incompletas por falta de información, sin importar qué LLM se use como generador.

**MRR (Mean Reciprocal Rank)** proporciona una perspectiva diferente pero complementaria: en lugar de preguntar "¿está el documento correcto entre los K primeros?", pregunta "¿en qué posición está el documento correcto dentro de los K?". Para cada query, el reciprocal rank es  $1/\text{posición del primer documento relevante recuperado}$  (1 si está en primera posición,  $1/2$  si está en segunda,  $1/3$  si está en tercera, 0 si no está entre los K). El MRR es el promedio de estos valores sobre todo el dataset. La diferencia con Recall@K es importante en la práctica: un sistema con  $\text{MRR}=0.85$  tiene el documento correcto en primera

o segunda posición en la gran mayoría de las consultas, lo que es relevante porque los modelos de lenguaje dan más atención a los primeros chunks del contexto —el "lost-in-the-middle" effect documentado por Liu et al. (2023) muestra que los LLMs tienen menor capacidad de atención en las posiciones medias de un contexto largo. Un retriever que consistentemente coloca el chunk correcto en quinta posición tiene  $\text{Recall@5}=1.0$  pero  $\text{MRR}\approx 0.2$ , lo que puede producir respuestas de menor calidad que un retriever con  $\text{MRR}=0.8$  aunque tenga el mismo  $\text{Recall@5}$ .

**NDCG@K (Normalized Discounted Cumulative Gain)** es la métrica más sofisticada y la más usada en evaluaciones formales de recuperación de información (benchmark TREC, BEIR). Su principal ventaja sobre  $\text{Recall@K}$  y  $\text{MRR}$  es que soporta **relevance gradada**: en lugar de clasificar cada documento como relevante o no relevante, permite asignar scores de relevancia en una escala (0=irrelevante, 1=parcialmente relevante, 2=muy relevante, 3=perfectamente relevante). Esto captura la diferencia entre recuperar un documento que contiene la respuesta perfecta (score=3) y uno que contiene información relacionada pero no directamente relevante (score=1). El cálculo tiene dos pasos:  $\text{DCG@K} = \sum(\text{relevance}_i / \log_2(i+1))$  suma los scores de relevancia descontados logarítmicamente por la posición;  $\text{NDCG@K}$  normaliza el DCG por el DCG ideal (el que se obtendría con los documentos perfectamente ordenados).  $\text{NDCG@K}$  requiere un dataset de evaluación con relevance gradada, lo que implica mayor esfuerzo de anotación que los datasets binarios relevante/irrelevante.

La **construcción del dataset de evaluación** es el prerequisite de todas estas métricas. El dataset debe contener: un conjunto de queries representativas de la distribución real de uso (factual simple, comparativo, multi-hop, temporal, ambiguo); para cada query, los `chunk_ids` de los documentos del índice que son relevantes (`qrels`); y opcionalmente, el nivel de relevancia de cada documento (para  $\text{NDCG}$ ). La anotación manual por expertos del dominio produce el dataset de mayor calidad; la generación sintética con LLM es más rápida pero introduce sesgos de homogeneidad. Un dataset de 100–200 queries bien anotadas es suficiente para la mayoría de los sistemas de producción, y es la inversión de evaluación mínima necesaria para tomar cualquier decisión de optimización con evidencia cuantitativa.

## Métricas de recuperación y sus casos de uso

- **Recall@K (K=1, 3, 5, 10, 20)**: métrica primaria para RAG;  $K=5$  es el estándar para sistemas que envían 5 chunks al LLM;  $\text{Recall@5} > 0.75$  es el umbral mínimo para producción;  $\text{Recall@20}$  mide el ceiling del retriever antes del reranking.

- **Precision@K**: fracción de los K recuperados que son relevantes; trade-off inverso con Recall@K; relevante cuando los tokens de contexto son limitados y cada slot tiene alto costo; Precision@5 > 0.60 indica que la mayoría de los chunks enviados al LLM son pertinentes.
- **MRR (Mean Reciprocal Rank)**: rango entre 0 y 1; MRR=1 si el primer resultado siempre es el correcto; complementa Recall@K con información sobre la distribución de posiciones del hit correcto; especialmente relevante por el "lost-in-the-middle" effect de los LLMs.
- **NDCG@K**: soporta relevance gradada (0/1/2/3); pondra logarítmicamente por posición; estándar en BEIR y evaluaciones formales; requiere anotación de relevance gradada, más costosa pero más informativa.
- **Hit Rate**: si al menos un documento relevante está entre los K recuperados, es un hit; idéntico a Recall@K con un único documento relevante por query; más fácil de explicar a stakeholders no técnicos; útil como métrica de comunicación externa.
- **Dataset de evaluación mínimo**: 100–200 queries representativas con qrels anotados manualmente o generados con LLM+validación; segmentado por tipo de query (factual, comparativo, multi-hop); la calidad del dataset determina la confiabilidad de las métricas calculadas sobre él.

Las métricas de recuperación son el lenguaje cuantitativo del retriever: sin ellas, cualquier discusión sobre "mejorar el retriever" es ambigua. Con ellas, las decisiones de optimización se vuelven experimentos con hipótesis medibles y resultados verificables. El capítulo concluye con la reflexión sobre el retriever como el cuello de botella más común en sistemas RAG de producción y los principios que guían su optimización sistemática.

---

## Módulo 6 – Capítulo 05 – Sección 06

---

### Cierre: la recuperación es el cuello de botella más común en sistemas RAG

La evidencia empírica acumulada en la industria es consistente y específica: cuando un sistema RAG produce respuestas de baja calidad, la causa raíz es la recuperación deficiente en la mayoría de los casos, no la generación deficiente. Esta asimetría tiene una implicación operacional directa: si se diagnostica baja calidad en el sistema RAG y el primer impulso es cambiar el LLM generador o mejorar el prompt, el equipo probablemente está atacando el componente incorrecto. La forma correcta de diagnosticar el cuello de botella es evaluar el retriever de forma aislada con el dataset de evaluación: si el Recall@5 es inferior a 0.70, cualquier mejora del generador está limitada por la información que el retriever no pudo encontrar.

La lógica matemática de esta asimetría es simple: el LLM generador solo puede producir respuestas correctas basadas en información que está en el contexto que recibió. Si el chunk con la respuesta correcta no está en los top-K recuperados —si Recall@K=0.60, eso ocurre en el 40% de las consultas—, el LLM se enfrenta a dos opciones igualmente malas: responder con información incorrecta del contexto recuperado (hallucination) o responder que no tiene información (abstención inapropiada). Ninguna cantidad de prompt engineering o de upgrade del modelo generador puede inventar información que no recibió. Por el contrario, incluso modelos de generación de capacidad moderada producen respuestas de calidad aceptable cuando el contexto recuperado contiene la información correcta y relevante.

Esta realidad convierte al diseño y la optimización del retriever en la primera prioridad técnica de cualquier proyecto RAG, y a las métricas de recuperación (Recall@K, MRR) en los primeros indicadores que el equipo debe monitorear en producción. Los sistemas RAG de mayor impacto en producción —los que mantienen calidad consistente durante meses y años— invierten desproporcionadamente en la calidad del retriever: chunking evaluado experimentalmente, modelos de embedding seleccionados con benchmarks internos, búsqueda híbrida como configuración estándar, reranking sobre los top-100 candidatos, y monitoreo continuo del Recall@K con alertas cuando cae por debajo del umbral. El

generador, que en estos sistemas suele ser un modelo de API estándar con un prompt bien diseñado, contribuye a la calidad de la experiencia del usuario, pero la fundación es siempre el retriever.

La arquitectura del retriever moderno de producción combina invariablemente: búsqueda vectorial con embeddings especializados en el dominio, BM25 o SPLADE para matching léxico exacto, fusión RRF para combinar ambas señales, reranking cross-encoder sobre los top-100 para elevación de Precision@5, y técnicas de transformación de queries para casos de uso donde el vocabulario mismatch entre usuarios y corpus es significativo. Este stack de retrieval, evaluado con Recall@K >0.80 sobre un golden dataset representativo, es el prerequisite para construir un sistema RAG de producción confiable. El Capítulo 6 abordará cómo evaluar el sistema completo de forma sistemática y continua.

*"Si no puedes encontrar la información, ninguna cantidad de inteligencia en el sistema la puede generar."* — Ellen Voorhees, investigadora del NIST y coordinadora del benchmark TREC durante más de 20 años, sobre la primacía del retrieval en sistemas de question answering.